第 9 章練習　虛擬記憶體

本檔只含題目。請先自行建立狀態或變數表，再作答；解答在獨立檔案。

OS09-E01　Reference string 替換

初始 frames 全空，主要比較容量為 3；reference string 為 1,2,3,4,1,2,5,1,2,3,4,5。比較 4 frames 時也從空狀態開始。CLOCK 的 slots 依 0、1、2 裝入，初始 hand=0；填入空槽不移動 hand，hit 時設該頁 R=1、不移 hand。滿載後從 hand 開始掃描，R=1 時清為 0 並前進，遇 R=0 則替換、設新頁 R=1，並將 hand 移至下一槽。OPT 在下次使用時刻相同（包含都不再使用）時，選頁號較小者作 victim。

1. 計算 FIFO、LRU、OPT 與 CLOCK 的 page faults。
2. 列出 FIFO 的 victim sequence，並指出哪些 hits 不改 FIFO queue。
3. 比較 FIFO 使用3與4 frames 的 fault count；是否出現增加？
4. Belady anomaly 為何不能推翻 OPT 的下界？精確 LRU 的硬體代價是什麼？

OS09-E02　有效存取時間

正常 memory access 為 80 ns；發生缺頁的單次存取從發生 fault 到完成重試的總成本為 5 ms，已含該次存取完成所需成本。p 是長期 fault 比例／單次存取的 fault 機率，並非固定每隔若干次一定發生一次；要求 EAT ≤ 100 ns。第 3 小題的 10 ms 也採相同總成本慣例。

1. 建立 EAT 不等式。
2. 求可接受的最高 p。
3. 若 fault service 提高到 10 ms，最大 p 如何變？

OS09-E03　Working set 與 thrashing

可用 64 frames；三個 process 在最近窗口內分別碰到 20、24、28 個不同 pages。

1. 以簡單 working-set 總和估計是否超過 frames。
2. 若超過，是否一定已發生 thrashing？
3. 系統可調整哪些控制量？

OS09-E04　Demand fault 與 COW fault 完整路徑

固定4 KiB pages、3 frames全滿：F0→P0 clean、F1→P1 dirty、F2→P2 clean。P5是合法readable VMA但nonresident，替換選F1；read與writeback各8 ms，兩次 I/O 不重疊、其他開銷忽略；假設 page lock 成功、無並行修改，且回收前已完成所有相關 CPU 的 shootdown。另在獨立的COW情境中，已合法取得可用frame F23（必要回收與舊引用處理已完成）；parent/child共享F17（x=41、refcount2、COW、read-only），child寫入42。

1. 列出read P5的逐步狀態、I/O數、TLB事件與service time。
2. 若victim改成clean P2，哪些數字改變？
3. child的write fault為何不是missing-page fault？列出F17/F23與兩張PTE的狀態。
4. 舉出兩個會造成infinite-fault或資料損壞的handler錯誤。

OS09-E05　Frame allocation 與 local/global victim

第1小題為獨立quota模型：64 frames分配給A/B/C，process sizes=100/200/300。第2至4小題另用6-frame全滿模型，與64-frame quota模型分開；現有last-use：A1=10、A2=5、A3=8、B4=2、B5=7、C6=9。A={1,2,3}、B={4,5}、C={6}，A fault page7且無free frame。

1. 以proportional allocation、floor後最大餘數分配剩餘frame，求最終quota。
2. local與global replacement各自選哪個victim？列操作後resident sets。
3. 哪個策略會把壓力擴散到B？提供一個立即後續fault。
4. 若某process小於minimum resident requirement，增加演算法公平性是否有用？

OS09-E06　Working set 與 PFF 控制

window Δ=4。A最近references=1,2,3,1,2,4,5,1；B=6,7,6,8,7；C=9,9,9。可用frames=6；此題A、B、C 頁面不跨 process 共享，總和 D 才能直接解釋為獨立所需 frame 數。PFF上界0.4、下界0.1；三個最近5-event window分別為 FFFFF、FHHHH、HHHHH。

1. 求A/B/C working sets、WSS與D；D>m表示什麼？
2. 計算三個PFF與控制決策；為何需要兩個門檻？
3. 若高PFF但沒有free frame，系統可做什麼？
4. 給出一個D>m卻不是thrashing的反例，並說明需哪些runtime證據。

OS09-E07　容量單調性與離線最優的證明

初始resident空、固定任意有限reference string，每次fault成本1且無prefetch收益；比較精確LRU與OPT。

1. 用與容量無關的recency stack證明LRU的resident inclusion，進而證明fault count不增。
2. FIFO或CLOCK為何不能直接沿用此證明？
3. 用第一次分歧的交換論證說明OPT移除下次最晚使用頁不損最優；指出dirty I/O成本改變時哪個前提消失。

OS09-E08　COW與舊TLB的隔離反例

F1 原為 P1 的可寫 mapping，CPU1 仍快取 P1→F1。另有獨立情境：P1／P2 各以 MAP_PRIVATE 映射同一 file-backed 4 KiB page F40，兩張 PTE 均唯讀並以 COW 管理，頁面與原檔初值為 7；沒有外部檔案寫入或其他並行寫入。第 3 小題再獨立假設 child COW 後 parent 指向私有 anonymous 頁，只有一筆使用者 mapping。

1. CPU0只清PTE就把P5載入F1，CPU1再store P1會造成什麼？給出正確先後順序。
2. 若只有P2寫9，列P1/P2 mapping、值、copy bytes與原file；再令P1寫8。
3. child COW後原頁只剩parent一個mapping，是否必須立即修改parent PTE？何時可免copy提升？

OS09-E09　Linux 3.2 reclaim的條件辨析

採本章v3.2 asynchronous order-0模型，kswapd=true、priority9、may_writepage/FS/I/O/unmap皆允許，page lock 均成功，無重新引用或並行修改，P1 非 lumpy reclaim；P2 是 clean file、無額外 pin 且 mapping／引用可移除；P3 是 dirty file、PTE reference 與舊 PageReferenced 均為 0，且非同步 pageout 成功啟動；P1 為 PTE referenced anon；P4 已被隔離、尚未完成 cull 的 mlocked 頁；P5 在 writeback 中。P4 的選取是隔離與轉移至 unevictable 的暫態，而非穩態 mlocked 頁仍位於普通 inactive LRU。

1. 逐頁列第一輪結果與free增量。
2. P3改direct reclaim或priority10會改哪個分支？
3. P1改成首次PTE reference的file且原PageReferenced=0，結果為何？
4. 把P4改成一般LRU中額外DMA pin，與mlock一樣嗎？writeback完成是否足以立即回收P3？

OS09-E10　視窗長度與 LRU 容量能互換嗎？

使用頁序列 1,2,1,3,1,2,4,1。工作集視窗 Δ=4；LRU 初始為空，頁等大，沒有預取或釘選。

(1) 列出 t=4 至 8 的視窗、WS 與 WSS。解釋 t=5 移出頁 1 的引用時，為何不能把頁 1 從 WS 刪掉。
(2) 列出每次存取前的 r，以及容量 2、3 的 F/H 與總缺頁數。
(3) 在 t=7 比較 WS 與三個 frame 的 LRU 駐留集合。以 d 的定義證明命中條件，說明它與 r 的差一關係。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 9. Linux v3.2 vmscan.c; Linux v3.2 memory.c; mmap(2).

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。
https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study